

Shazaming Whales: Using Convolutional Neural Networks to Analyze Whalesong

Motivations:

All things communicate on some level, especially those of the same species. For whales, the majority of them communicate using long, low-frequency sounds that are able to travel long distances through the waters of the ocean. These sounds are also useful for navigation, as both sight and smell are limited at the depths that whales typically live. The low-frequency noises are often called “whalesong,” due to their pattern and predictability across species. All the whale species we looked at for this project are considered endangered, so being able to classify their sounds can be vital to their preservation (NOAA Fisheries). Being able to classify the sounds whale species make can help determine population counts, map migration patterns, and relationships and interactions between members of various pods.

Background:

In the initial set of data we were given, there were 6 different whale species detected. The initial whale species were the Blue Whale, Humpback Whale, Multiple-Humpback Whale, Sei Whales, Fin Whales, and North-Atlantic Right Whales (NOAA). We later discuss why not all the whale species were utilized, but for now its important to talk about the three whale species we made use of. The first whale we used data for is the Fin Whale. The Fin Whale is the second largest of all whale species, weighing an average of 40 to 80 tons. They are found in all of the worlds oceans and they hold an endangered status (Fisheries). The second whale we looked at with this data is the Humpback Whale. A favorite of whale watchers, this species enjoys breaching from the water to make a dramatic splash. The Humpback Whale is another species that is found in most of the world's oceans and it also holds an endangered status (Fisheries). The

final whale we looked at is the North-Atlantic Right Whale. The North-Atlantic Right Whale is one of the most endangered whale species, with only about 370 remaining in the world. During the peak of commercial whaling, this species was particularly targeted due to them being the “right” whale to hunt due to the fact they float once killed. This species of whale can be found on the Atlantic coast of North America and the Atlantic coast of Europe. Though all these species are considered endangered, there have been significant increases in their populations since the reduction of commercial whaling in 1986 (Fisheries).

This project aims to take raw sound data from a stationary marine recording device located in the North Atlantic Ocean and train a neural network on confirmed detections of three species of whales. We pull our data from NOAA’s Passive Bioacoustic database, from dates between March 28th, 2009, and April 03rd, 2009, as well as a database of confirmed detections of whales from that period of time in order to train the neural network on confirmed data (NOAA).

Data Wrangling:

The bioacoustic data in its initial form from the NOAA National Centers for Environmental Information database was separated into 672 fifteen-minute sections beginning on March 28th, 2009, at midnight and continuing through April 04, 2009, at midnight. Additionally, we had access to a dataframe that contained the information about confirmed detections from this time period. The data contained the start time and date, and the end time and date of the specific song, maximum and minimum frequencies during that time period, as well as the species and information about the detection. This information is what we used to separate the sounds into confirmed detections in order to train our model.

In the initial data set, there were six types of whales that had detection information: North Atlantic Right whales, Sei whales, Humpback whales, Humpback pods, Fin whales, and Blue whales. For both Sei and Blue whales, there were a very limited number of detections of the whales, to the point where even if we upsampled the data, it would cause the model to overfit to the duplicate sounds and result in a faulty model. Additionally, for both of those whales, there were no confirmed detections, only possible detections, so we would not have been able to train the model on confirmed detections from those species. We also cut out the Humpback pods, because we were concerned that the model would have difficulty separating the singular Humpback whales from the groups.

Our initial plan to wrangle the data involved turning each fifteen-minute section into a list of frequencies, then cutting the data based on the times provided in the detections dataframe. This quickly proved to be a difficult task, due to the sheer amount of data in each segment. Each file was approximately nineteen million datapoints, due to the sample rate of the audio file being over 22,000.

We attempted to downsample from the nineteen million data points to only 45,000 using a Python module called aubio to convert the audio into frequencies. We then attempted to assign a datetime value to each datapoint, as each point was equidistant. However, when we compared the maximum and minimum frequencies of the segments where whales were detected to the values provided in the dataframe, we found a large discrepancy, and thus discarded this method.

Because the frequencies are what are used to determine which whale species is being detected, we still needed to figure out a way to extract that data from the sound files. At this point, working with the individual fifteen-minute files had become cumbersome, so we combined the data into seven 24-hour files, one for each day the data was taken. This also made

it easier to separate the data into sound segments where there were confirmed detections. After the separation was done, we converted the segments into spectrographs in order to train the model.

From here, our plan was to combine the 672 15-minute segmented audio files into 7, 24-hour audio files to accurately match them up to the time detections in the dataframe to cut the individual clips needed for each species. In order to combine the 672 files, we utilized a package called PyDub. Pydub allows audio files to be read into the terminal, and we created a code that concatenated all audio files together. This resulted in the 7 separate 24-hour audio files.

Data Manipulation:

In order to separate the files, we utilized the datetime module to transform the start time and end time of each detection into milliseconds from midnight, then iteratively ran through each day and species to separate the audio into the correct categories. For each whale species, we decided to obtain 1,000 audio recordings per species to train the neural network on.

CNNs run most effectively when their inputs are all the same size. If we were to use audio files of different sizes, the CNN would struggle with extracting features from the spectrograms. This would cause vital information to be potentially cut or misread, leading to inaccuracies in the final model (Nandi). Due to the varying lengths of time of detections, we decided to pad every audio file to be 30 seconds by adding zeros to the end of each file. There were only a handful of clips over 30 seconds, so we decided to cut the excess time, as the length would capture the majority of the whalesong recorded to the point that the species would still be identifiable (Nicholas). Since there were only a few clips over 30 seconds, we determined that it would not alter the results of the data to a significant degree if it were to be cut. PyDub was

utilized to run a code that would read all of the files, determine their length, and pad or cut their length to 30 seconds. This resulted in the 3,000 audio files all having the same length and size, which makes them ready to be trained on the CNN model.

Model 1: Binary Classification Between Whale Sounds and Ocean Noises

At first, we just wanted to create a model that would classify if the sound we gave it was a Fin Whale, Humpback Whale, or North Atlantic Right Whale. However, we had concerns that we would not be able to successfully train that model. We decided to shift gears to do a simple classification of whether a sound we gave the model was a whale or not a whale. This required searching for passive acoustic data that was not specific to detecting whales. We found an open collaboration dataset by the International Whaling Commission's (IWC) Southern Ocean Research Partnership (SORP). This data was collected over the course of 10 years across several sites to monitor acoustic activity in the ocean (Miller et al.). Throughout the audio, you can hear a wide variety of aquatic species and ships passing by the acoustic technology. We took this data and followed the same steps used when cleaning the whale audio data, creating 3,000 30-second audio files of miscellaneous ocean acoustics sounds. At this point, we had all the data we needed to convert the segments into spectrograms to train the model.

The most important part of making sure that our audio files are correctly converted into a usable format is to make sure they are read into our environment correctly. This is done using a Python package called *Librosa* with its function “`librosa.load()`”, which allows the specific audio file to be converted into a mel spectrogram. Mel spectrograms are visual representations of frequencies over the length of the file we are analyzing. With this, we were able to create a function that would not only convert the audio files into mel spectrograms but also standardize,

normalize, and resize them into a valid input for a CNN. This worked for the most part; however, after reviewing the code over again, the audio files were all standardized into one-second clips, possibly cutting out some whale calls that were present after the audio was finished. We normalized the data by subtracting each data point from the mean and then dividing the difference by its standard deviation. This allows for each data point to be read as a value between 0 and 1, allowing for less bias within the model training process. Finally, we then resized each audio file into 128x128 pixels for the CNN to take in each spectrogram at a reasonable size. This would allow the model to actually “see” the spectrograms in the best detail possible. This can be done similarly for both of the models we trained on this dataset.

Creating the CNN model for the binary classification between ocean noises and whale calls was fairly simple. We had our data organized into two folders: one with padded audio files of whale noises at 30 seconds each, whether there was any blank noise or not, and another with various ocean sounds that did not have any sort of whale detection previously attached to them. This allowed us to have two completely separate audio paths and 6000 audio files (3000 files for each label) for the model to take in as input. The data was trained on 80% of the audio files we had compiled, and 20% was left over for validation metrics. We decided to keep this same split in our second model for consistency. We then decided to have enough layers so that the model would be able to accurately process the mel spectrograms. We had three Conv2D layers that had 32, 64, and 128 units, respectively, and two dense layers with one containing 64 units and the output layer of 1 unit. The output was normalized using a Sigmoid function for the binary probability classification, and binary cross-entropy was utilized as our loss function. Over time, we decided to include dropout to reduce the risk of overfitting and to stop at a point where the model will no longer improve over time. In addition, we included early stopping to couple with

dropout, as well as a reduced learning rate adapter that would change the learning rate over a certain number of epochs.

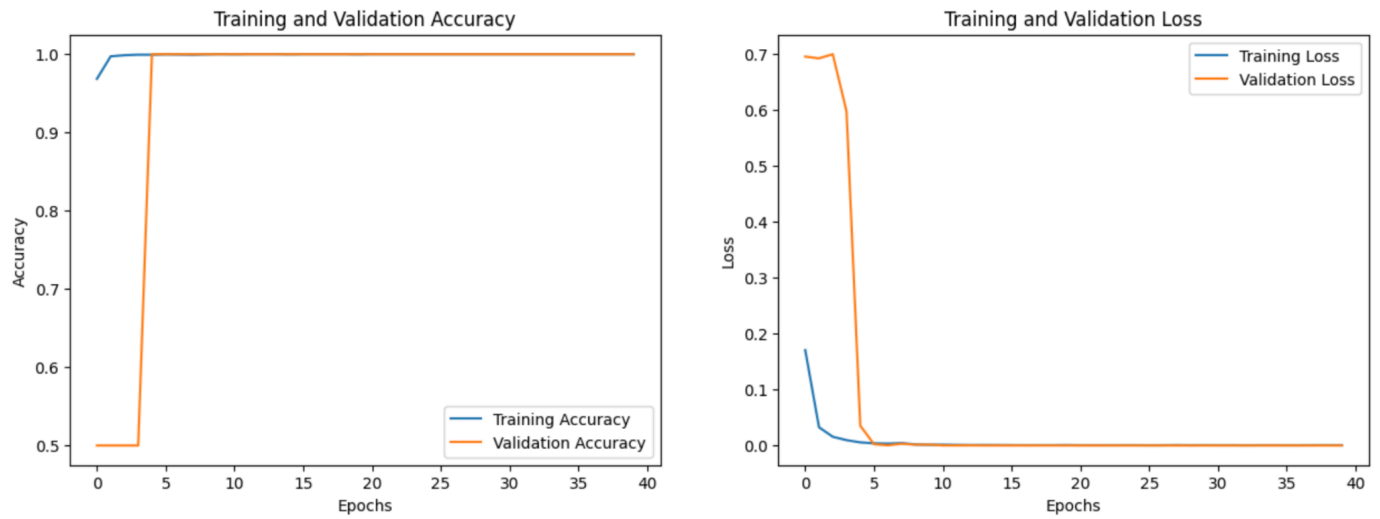


Figure 1: Training and Validation Accuracy and Loss Curves

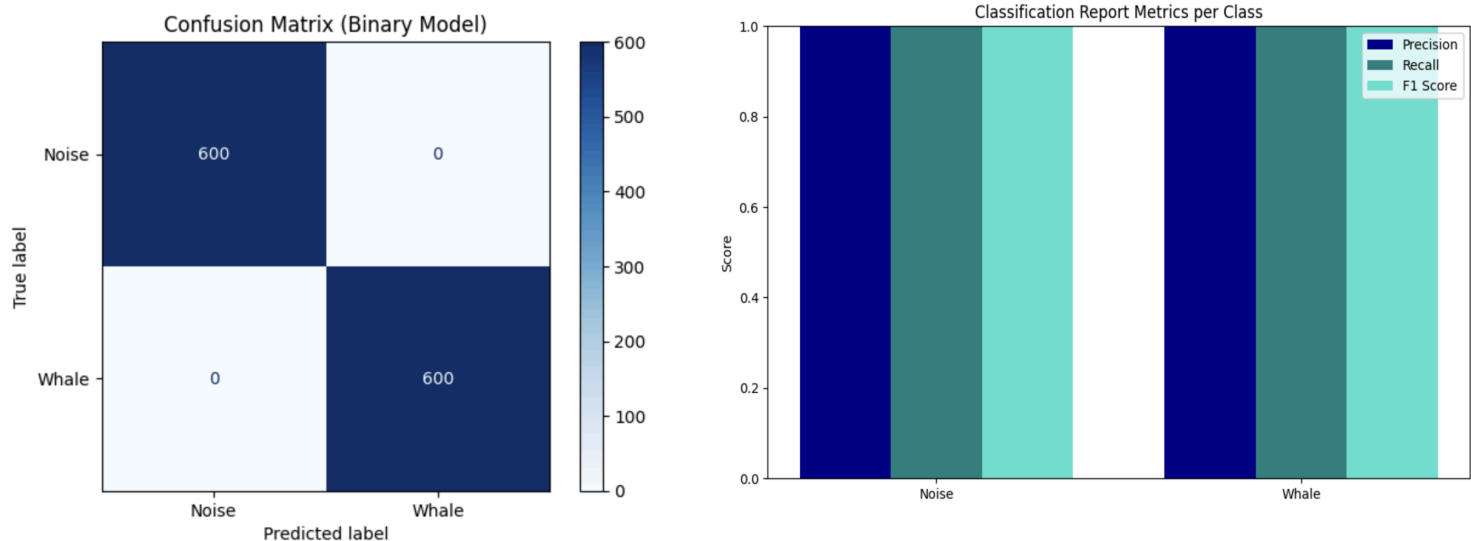


Figure 2: Confusion Matrix for Binary Model

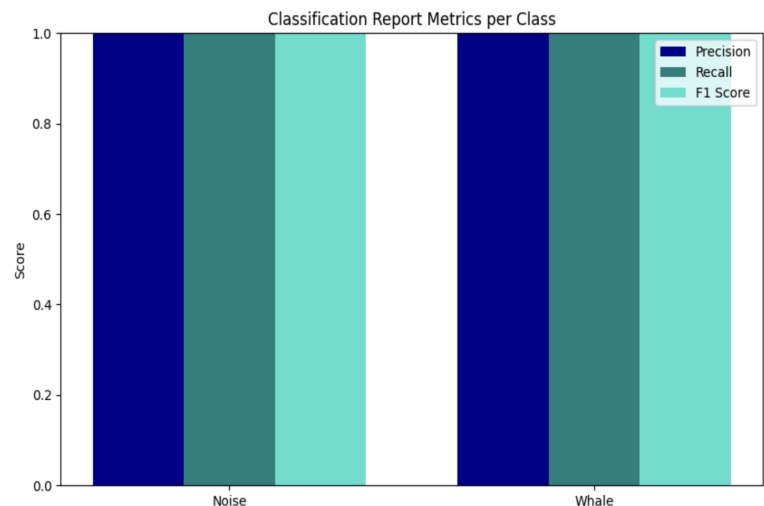


Figure 3: Classification Report for Binary Model

The results for the binary model appear to be a perfect representation of the data, which is not typical behavior of CNNs. The reason is that this type of behavior shows that the model is not learning the data but rather memorizing it, making it inadequate for any real-world

application. The cause was unknown until we looked back at the original spectrogram images we preprocessed.

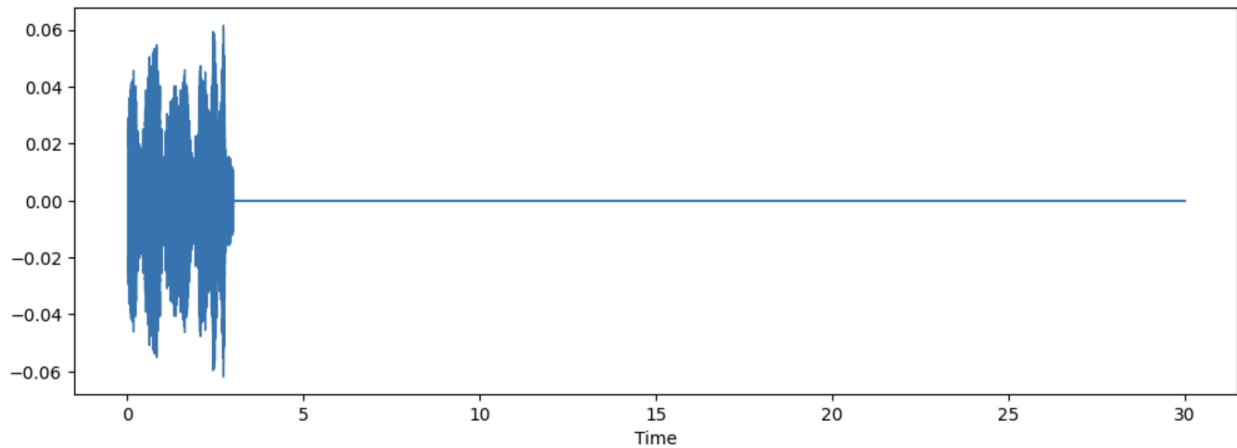


Figure 4: Spectrogram of Humpback Whale Call

This mel spectrogram is a visual representation of what one would expect to see coming from the padded audio file folder we utilized: a couple of seconds of audio showing the whale call, and the rest being padded with blank noise.

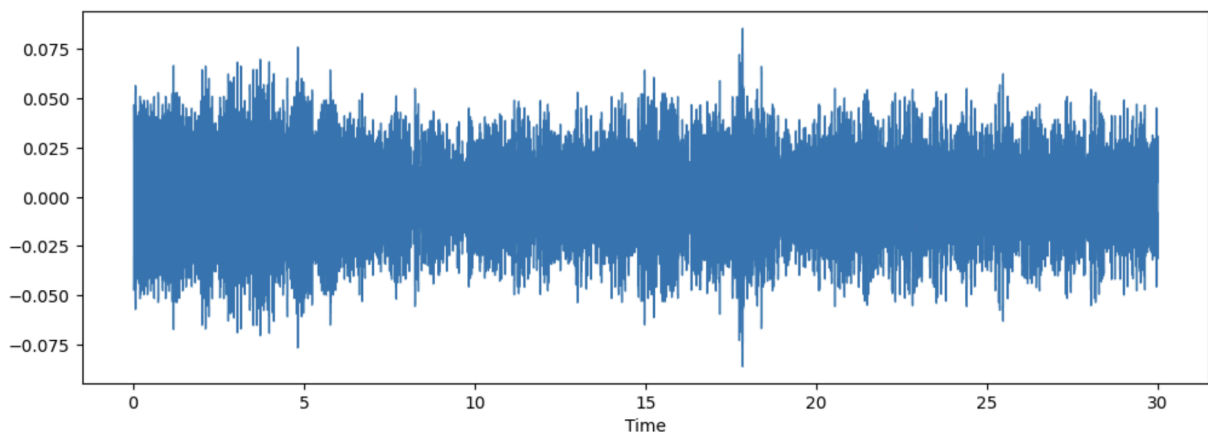


Figure 5: Spectrogram of Ocean Noise

Compared to the one above, this mel spectrogram has much more information within it and shows a full 30-second clip of ocean noise, giving various frequencies with no blank noise.

Because of this vast difference, the model was memorizing the numpy array dimensions of these spectrograms and classifying them 100% correctly, leading to a model that would not apply to a real-world scenario.

Model 2: Three-class Classification of Various Whale Species

After compiling a model that would, without the data labeling error, classify between two sets of labeled data, we decided to tackle our second challenge of correctly classifying between audio files of Humpback, Fin, and Right Whales. The same pre-processing was done to convert the audio files into spectrograms for inputting into the model itself. This was done with the 3000, whale sounds with padding that we had used in the previous model for whale vs noise detection. For the model's structure, we decided to keep the three Conv2D layers with two dense layers, but we had to change the last activation function as we are no longer dealing with binary classification. With this, the last activation function is a softmax function, which allows for a vector of probabilities for each class to match with our model's output. We also had to change the loss function to a sparse categorical cross-entropy function, allowing for multiple classes to be identified and categorized based on the spectrograms. Lastly, we also modified the dense layers to have one dropout layer at 50%, and we included early stopping and a reduced learning rate on the plateau function for callbacks. All of these lead to a better-classifying model than the previous one.

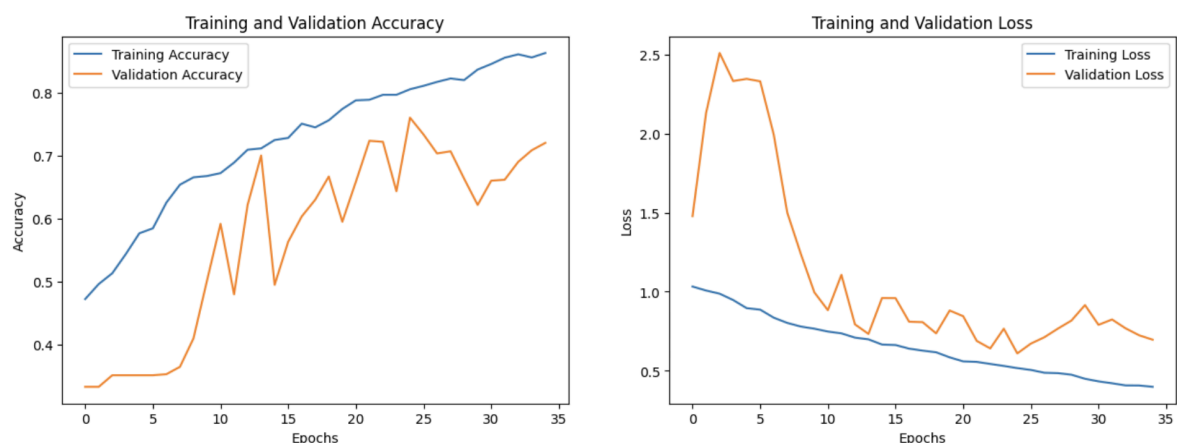


Figure 6: Training and Validation Accuracy and Loss Curves

The following curves show a gradual progression of the model learning from the data. With final accuracy scores at 85% for training and 72% validation accuracy, the model did a fairly good job at classifying all of the different types of whales based on their frequencies. Though the curves did not fully converge, the area under the curve is not very high, showing that the validation accuracy is a fair representation of how well the model learned from the audio files.

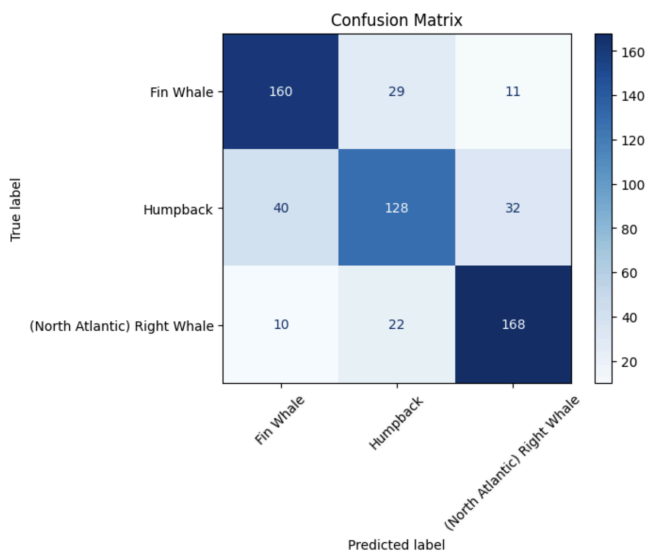


Figure 7: Confusion Matrix for 3-Class Model

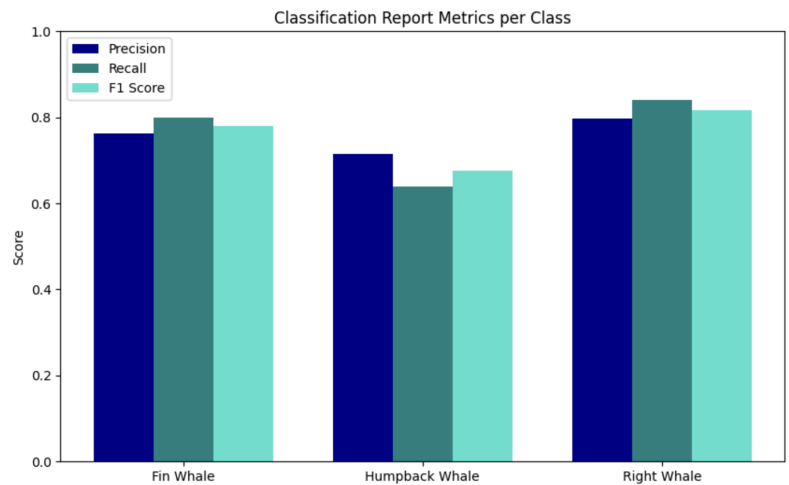


Figure 8: Classification Report for 3-Class Model

The confusion matrix and classification reports show a much better spread of the metrics. The right whale had the highest precision, recall, and F1 scores, all around 80%. With this, this model only had some slight oscillation problems with the accuracy and loss curves, resulting in a fairly good model for classifying the various whale types.

Conclusion and Future Work

Some improvements could certainly be done and continued with these models. Due to computational resources, having ample amounts of data was a struggle, as CNNs learn the best

by having as much data as possible. We had to limit our data to 3000 files for all of our whale species sounds due to the worry of not having enough time or processing power to compute all of the spectrograms and train a model that can take high amounts of data as input. Secondly, when comparing the whale calls to the ocean noise, the spectrogram patterns were vastly different, leading to a complete memorization of the data rather than learning and being able to apply it to unseen validation sets. With this, comparing the whale calls to another low-frequency sound with lots of variance, such as ships or dolphins, would benefit and give the model more of a challenge to learn from the data, rather than copying it completely. Lastly, extending this project into a bigger time period, rather than around a month, would benefit significantly due to the need for research into CNNs for those who are unfamiliar with the concept or are not as experienced with building these types of models in a short amount of time.

Convolutional Neural Networks (CNNs) are always changing and provide frequent insight into the world of machine learning and artificial intelligence. With its ever-changing documentation and application into the real world, learning how to utilize these models in real-world scenarios can assist many individuals in research-heavy fields for accurate predictive data. For this project specifically, this research will allow marine researchers to gain more insight into whale migration patterns, activity, and population frequencies. As the world of technology improves, it is up to the world to decide how it will apply it to the real world and continue to make its mark on progressive change.

References:

1. NOAA National Centers for Environmental Information. 2017. Passive Acoustic Data Collection. NOAA National Centers for Environmental Information.
<https://doi.org/10.25921/PF0H-SQ72> November 19, 2025.
2. Putthewad, R. B. 2024. Sound classification of Animal Voice. Kaggle.
<https://www.kaggle.com/datasets/rushibalajiputthewad/sound-classification-of-animal-voice/data>
3. Fisheries, N. (n.d.). Species directory. NOAA Fisheries. <https://www.fisheries.noaa.gov/species-directory>
4. Team, K. (n.d.). Keras Documentation: Keras 3 API documentation. <https://keras.io/api/>
5. Nandi, P. (2025, January 21). *CNNs for audio classification*. Towards Data Science.
<https://towardsdatascience.com/cnns-for-audio-classification-6244954665ab/>
6. Nicholas Renotte. (2022, April 16). *Build a Deep Audio Classifier with Python and Tensorflow* [Video]. YouTube. <https://www.youtube.com/watch?v=ZLIPkmmDJAc>
7. Miller, B.S., Milnes, M. and Whiteside, S. (2024) Long-term underwater acoustic recordings in the Southern Ocean 2013-2023, Ver. 8, Australian Antarctic Data Centre - doi:10.26179/qnn4-z957, Accessed: 2025-12-01
8. ricrash. (n.d.). *GitHub - ericrash/Old-Project--DSC-541-Dolphin-Classification-Project: Process boat audio and dolphin signals, then classify each as being a dolphin or a ship sound*. GitHub. <https://github.com/ericrash/Old-Project--DSC-541-Dolphin-Classification-Project/tree/main>